

Current FYP Features

- **Hybrid Ingestion:** Processes both local source code and remote GitHub URLs.
- **Static Analysis:** Integrated **OpenGrep** for fast, pattern-based vulnerability scanning.
- **Container Auditing:** Uses **Hadolint** for Dockerfile linting and security best practices.
- **Secret Detection:** **Gitleaks** integration to find hardcoded credentials and API keys.
- **Vulnerability Scanning:** **Trivy** implementation for CVE and misconfiguration detection.
- **AI Reasoning:** Intelligent analysis to identify complex flaws that static tools miss.
- **Dynamic Patching:** Automated generation of security fixes for identified issues.
- **Sandbox Verification:** Secure environment to test and validate patches before deployment.

ALREADY IMPLEMENTED FEATURES

Category 1: Offensive "Attacker" Features (12 Features)

1. **Vulnerability Chaining:** Linking multiple small bugs to create one massive, successful attack.
2. **Business Logic Testing:** Finding flaws in "how an app works" (e.g., skipping a payment) rather than code errors.
3. **Automated Exploit Generation (PoC):** Writing a custom script to prove a bug is real and dangerous.
4. **Lateral Movement Simulation:** Testing if an attacker can jump from a hacked server to others in the network.
5. **Multi-Agent Swarm Testing:** Coordinating a "team" of specialized AI agents to hack a single target.
6. **Adaptive Fuzzing:** Sending "random" data to a system to force it to crash or reveal secrets.
7. **Identity-Based Attack Paths:** Testing if stolen employee logins can be used to reach sensitive data.
8. **Proof-of-Exploit Evidence:** Automatically taking screenshots or videos of a successful "hack."
9. **Dark Web Correlation:** Checking if found passwords or usernames are already leaked on the dark web.
10. **GraphQL & API Deep-Dives:** Performing complex security tests on modern, hidden API structures.
11. **Continuous Attack Monitoring:** Probing the system non-stop as code changes, rather than on-demand.
12. **Environmental Contextualizing:** Identifying if a bug is on a "Live" vs. "Test" server to prioritize risk.

Category 2: Automated "Fixing" Features (6 Features)

1. **Autonomous PR Remediation:** Automatically opening a Pull Request on GitHub with a code fix.

2. **Auto-Merge Remediation:** Merging the security fix automatically if it passes all original app tests.
3. **Regression Testing for Fixes:** Running the app's old tests to ensure a "patch" didn't break a feature.
4. **Batch Fixing:** Applying the same fix to dozens of similar bugs across different files at once.
5. **Zero-Trust Policy Builder:** Writing the specific firewall rules needed to block a newly found threat.
6. **AI-Powered "One-Click" Fixes:** Directly fixing bugs within the developer's code editor (IDE).

Category 3: AI-Specific Security (6 Features)

1. **Prompt Firewall:** Real-time blocking of "tricks" designed to hack an integrated AI bot.
2. **AI Model Sandboxing:** Running AI models in a safe, isolated box to prevent them from hurting the host.
3. **Prompt Injection Scanning:** Searching for ways a user could force an AI to ignore its safety rules.
4. **Model Integrity Monitoring:** Detecting if an AI's "brain" (weights) has been secretly tampered with.
5. **AI Risk Detection:** Identifying if training data used for a model was "poisoned" by an attacker.
6. **AI Security Fabric:** A unified dashboard that governs the security of every AI agent in a company.

Category 4: Analysis & Efficiency (8 Features)

1. **Reachability Analysis:** Checking if a bug can actually be hit from the internet to filter out "dead code" alerts.
2. **Context-Aware Triage:** Learning from your past decisions to stop flagging bugs you've marked as safe.
3. **Business Impact Scoring:** Explaining risk in terms of "money lost" rather than just technical scores.
4. **Shadow API Discovery:** Finding undocumented or "forgotten" web addresses that are wide open.
5. **Secret & Token Scanning:** Searching through code history for leaked passwords or API keys.
6. **Infrastructure-as-Code (IaC) Scanning:** Checking cloud setup files (like Docker or Terraform) for leaks.
7. **Human-in-the-Loop Supervision:** Allowing a human to "approve" the AI's next hack before it happens.
8. **Self-Learning Triage:** An AI that hides noisy alerts based on how "real hackers" would ignore them.

Advanced MAS & 2026 Security Features

1. **Red vs. Blue Wargaming:** Simultaneous attacker and defender agents that compete to find and patch vulnerabilities.
2. **Agentic Consensus Scoring:** Multiple specialized agents (Logic, Network, etc.) must "vote" to confirm a bug, reducing false positives.
3. **Hierarchical Orchestration:** A "Manager" agent that analyzes the tech stack and spins up specific "Worker" agents (e.g., for Hadolint or Gitleaks).
4. **Honey-Agent Deception:** Deploying fake "vulnerable" APIs and agents to trap and log manual attacker techniques.
5. **Post-Quantum Crypto Auditing:** Identifying legacy encryption (like RSA) and suggesting Quantum-Safe (PQC) replacements.
6. **SaaS-to-SaaS OAuth Worm Detection:** Scanning for identity-based lateral movement across integrated platforms like GitHub and Slack.
7. **PBOM (Prompt Bill of Materials):** Tracking and auditing every AI prompt to prevent injection or goal hijacking.
8. **Sleeper Agent Sanitization:** Regularly "wiping" agent memory to prevent long-term corruption or history poisoning.
9. **Predictive Behavioral Hunting:** Detecting "logic deviations" that suggest an active, unknown exploit rather than just known CVEs.
10. **Self-Healing CI/CD:** Automatically blocking and rolling back builds if a "poisoned" dependency is detected in the supply chain.